

ESE 205 Final Report

Autonomous Navigation and Speed Control of a Raspberry Pi-Powered Car Using PID Controllers and Sensor Integration

Jackie Wu, Joshua Yeh

Abstract - This project investigates the implementation and performance of a Raspberry Pi powered car capable of autonomous navigation, object tracking and consistent speed control. By using a Proportional-Integral-Derivative (PID) controller and different sensors, we were able to achieve acceptable speed regulation and drive the car with minimal overshoot. Despite challenges in noise and hardware calibration, the results showed the effectiveness of our PID control and color tracking program. Future improvements could include more advanced control and image detection algorithms.

I. INTRODUCTION

Understanding robotic systems is important in the modern world and has many applications from autonomous vehicles to industrial automation. This project utilizes a Raspberry Pi and different sensors to track objects, navigate autonomously and run at a consistent speed with minimal overshoot and error.

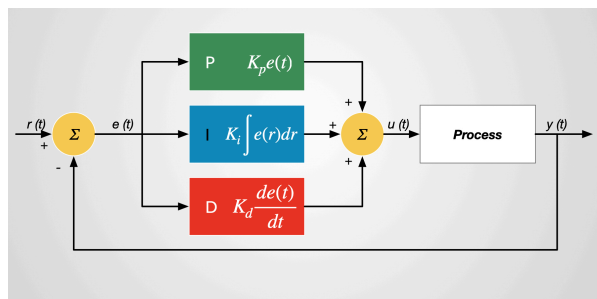


Fig. 1. Diagram of PID control. [1]

Proportional-Integral-Derivative (PID) control is an important technique in the control of the motor system. The PID controller works by continually calculating the error values and applies corrections based on these error values. These corrections are adapted using three terms. The proportional term

reacts to the current error, the integral term addresses cumulative errors from past data, and the derivative term predicts future errors. This process can be seen in Fig. 1.

This experiment demonstrates how PID control can optimize low cost robotic systems for practical applications. To determine the correct PID control, the following equation was used.

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de}{dt} \quad (1)$$

Where $u(t)$ is the PID control variable, K_p is the proportional gain, $e(t)$ is the error value, K_i is the integral gain, de is the change in error value, and dt is the change in time. The PID control equation aids in determining the past, current, and future error corrections and allows consistency in our motor speed.

II. MATERIALS

In this experiment, the following equipment was utilized and implemented. An ultrasonic sensor was used to measure the distance from the car to an object. A camera was used to track blue and yellow objects. A DC motor and gearbox was used to move the car. A photoresistor and LED were used to measure the rotations of the car wheels. Multiple servos were used to control the steering and the orientation of the camera and ultrasonic sensor. Finally, a Raspberry Pi and power supply hat were used to control and power the systems of the car.

III. CAMERA AND AUTOMATIC NAVIGATION

Within this part, objective two and objective four is going to be covered. To automatically navigate through the hallway, both objectives utilized the

camera and ultrasonic sensor. Specifically, an ultrasonic sensor is used to control the speed of the car and when to stop, and the camera is mainly used for steering.

Before diving into the control for sensors, the timing technique used in both objectives will be introduced. Instead of using a `time.sleep()`, a new timing approach is utilized. In specific, each task operates on its own schedule, controlled by a specific counter and associated delay. The counters (`counter_key`, `counter_dist`, `counter_camera`) increment after each execution of their respective tasks, and the condition

```
start_time+counter*delay < cur_time
```

This ensures that a task runs only when the appropriate amount of time has elapsed. For example, the distance measurement task reads data from the ultrasonic sensor at regular intervals of `delay_ultra`, adjusting the motor speed based on the distance reading. Similarly, the camera task processes images at intervals of `delay_camera` to track the blue tape, while the key input task checks periodically for user input to stop the car. This non-blocking approach allows the system to manage multiple tasks concurrently, maintaining real-time responsiveness and flexibility. Unlike traditional blocking methods, this implementation ensures precise timing for each task, crucial for the smooth operation of a multi-sensor control system.

Back to the control of the ultrasonic sensor, this project used a simple relationship between PWM and distance to control the speed of the car. For every 0.1 seconds, the ultrasonic sensor updates the `dist` variable, which measures the distance between the car and the nearest obstacle in front of it. Then, using the following relationship, the PWM for the car motors is updated.

$$pwm = (dist - 10) / 5 * 5 \quad (2)$$

Figure 2 demonstrates the workflow for the ultrasonic sensor.

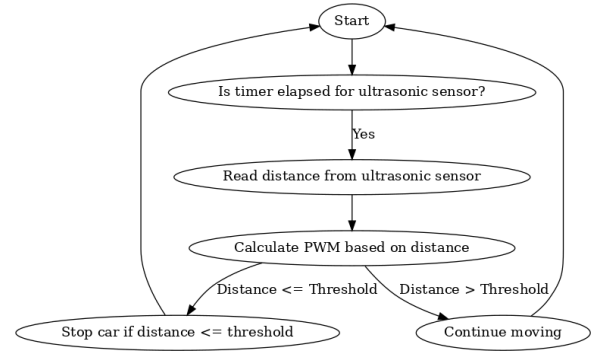


Fig. 2. Flowchart for ultrasonic sensor program.

The camera in the two objectives helps the car change directions and keep it on the right track. As introduced in the previous part, camera control also utilizes a counter based timing approach. Every 0.07 seconds, the camera captures an image, which is then converted from the BGR to the HSV color space to facilitate color-based object detection. A mask is generated using a predefined HSV range that isolates the target object, and the mask is further smoothed with a blur operation to reduce noise. The code applies thresholding to create a binary image, highlighting the object for easier processing. The object's position is determined using image moments, which calculate the centroid coordinates of the detected area. If no object is detected, the loop continues without adjustments. For a detected object, the `pos2angle()` function calculates the angle between the car's current orientation and the object's position, which is used to compute a new duty cycle for the servos. This duty cycle, clamped between predefined limits, ensures precise adjustments to the car's steering and swiveling, enabling smooth alignment with the object. Debugging information, including the object's position, angle, and annotated images, is logged for analysis and visualization. This entire process operates in a real-time loop, allowing the car to continuously track and navigate toward the object dynamically, demonstrating robust performance in tasks such as line following or object tracking. Figure 3 demonstrates the work flow for the camera.

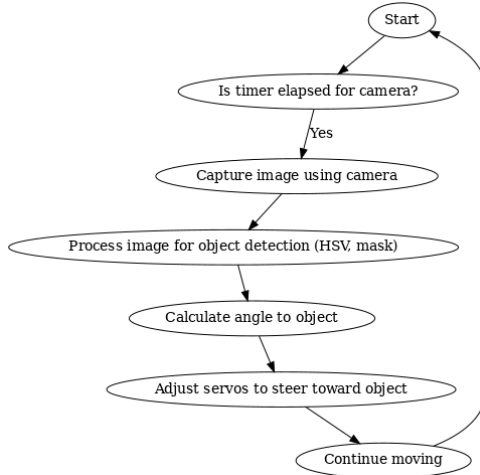


Fig. 3. Flowchart for camera steer program.

Specifically for objective four, a boolean variable `flag` is used. The `flag` is initialized to 0. When the car approaches the blue object and the distance is under the threshold, `flag` is changed to 1. And when that happens, the HSV values in the camera part is going to change from tracking blue into tracking yellow. And eventually, the car will be able to track the yellow object and stop in front of that.

In the future, the camera-based tracking system could be improved, making it more robust, accurate, and adaptable to different environments. One improvement would be to use more advanced computer vision techniques, such as machine learning models, to detect and track objects more reliably, even with variations in color, shape, and lighting. Another idea is to implement a dynamic HSV range adjustment algorithm to help the system adapt to changes in ambient light. Adding multi-object tracking could also be useful, allowing the car to identify and prioritize multiple targets or navigate more complex paths. Increasing the image resolution and refining the image processing pipeline would reduce noise and make centroid calculations more precise. I could also add other sensors, like LiDAR or depth cameras, to provide additional spatial information and work alongside the camera for better navigation. Finally, incorporating predictive control methods would make the car's movements smoother

and more efficient by anticipating where the object will be, instead of just reacting to its current position.

IV. MOTOR AND MOVEMENT CONTROL

This section covers objective 1 and 3, where the control and movement of the Pi-car are tested. For objective 1, the PID control for the car was tested. While the car was suspended with no load, we were able to determine the response time, overshoot and steady state error for the motor running at 3 rotations per second (RPS). We were able to determine the relationship between the PWM and RPS values by sampling different PWM values and measuring the RPS.

In objective 3, the user controlled the car movement while the car maintained a consistent speed and stopped after a certain distance. To do this, keyboard inputs were connected to the steer servo positions to allow for user control while the car was in motion. The system also used an encoder for speed control and an ultrasonic sensor to stop the car if an object was detected in front of it. The PID controller was adjusted to ensure accurate and consistent speed throughout its operation. This real time feedback allowed the car to adapt to different changes in the environment. This led to smooth deceleration and accurate stopping. This objective demonstrated the car's ability to integrate feedback control.

To determine the influence of K_p on a system, K_p values that were half and ten times our "best" K_p value were tested. K_i was set to zero for this section and the data was recorded and plotted.

V. RESULTS

The overall system performed as expected, meeting the requirements for each objective. Challenges included issues with the hardware steer and swivel servos and calibration issues with our car. These challenges were mitigated by calibrating the car every time before use.

For objective 1, the PWM and RPS relationship was found using a line best fit of speed at different PWM values. The results can be seen in Fig. 4.

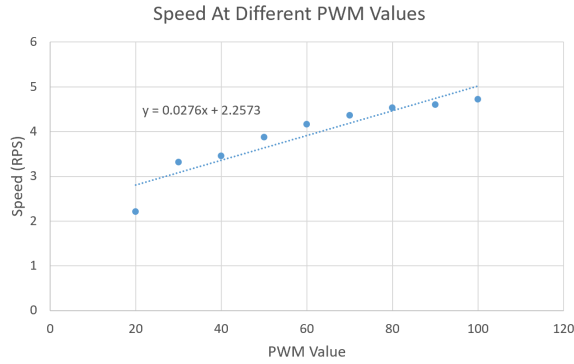


Fig. 4. PWM and RPS relationship.

When sampling the AD values from the photoresistor and velocity calculation, the AD values were sampled at a rate of 50 Hz and the speed calculations were sampled at a rate of 4 Hz. The higher sampling rate for the AD values allowed for changes in light intensity to be detected quickly by the photoresistor. This ensured accurate and real time detection of motor rotation. For the speed calculations, a lower sampling rate was chosen to minimize the load on the system while maintaining accurate data values for the car's speed. This sampling rate balances system efficiency with necessary data. These two sampling rates contributed to the system's ability to maintain stable and accurate motor control.

The PID controller's performance was evaluated when there was no load and stationary and when it was moving with a load. For the no load condition, the car had a rise time of 1.26 seconds to reach ninety percent of the target RPS of 3.

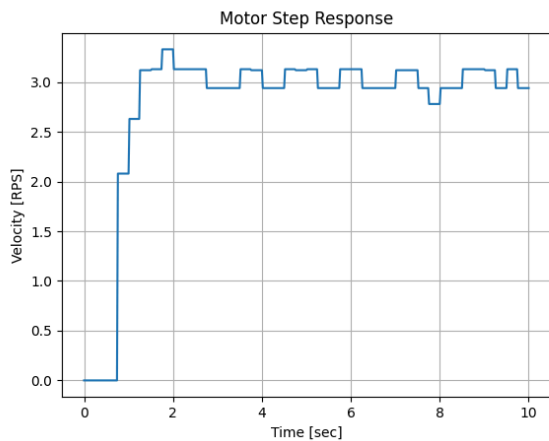


Fig. 5. Motor Step Response for 3 RPS stationary, no load car.

The motor response showed little overshoot and the steady state error was 0.3 RPS. These results show the effectiveness of the PID in a stationary and unloaded scenario. This graph can be seen in Fig. 5. For the stationary, unloaded scenario, there was zero delay, the K_p value was set to 8, the K_i value was set to 0.5, and the K_d value was set to 0.5.

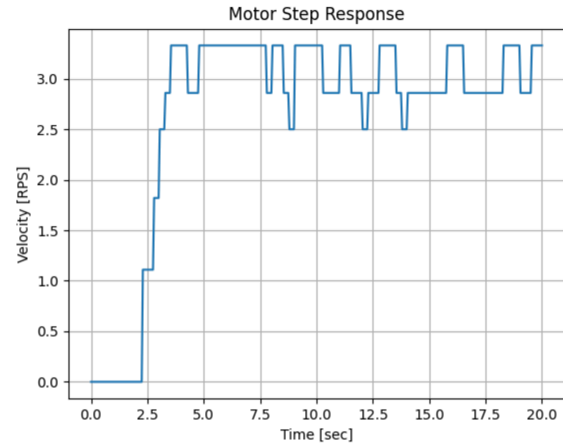


Fig. 6. Motor step response for 3 RPS moving, loaded car.

For the loaded moving car, the system achieved a rise time of 1.8 seconds, this is due to a set delay of 1.5 seconds before the car starts. The motor's response showed an overshoot of 11 percent and the steady state error of 0.07 RPS. These results can be seen in Fig. 6. These improvements show how adapting PID parameters can enhance the control in different situations. For the moving loaded car, the delay was set to 1.5 seconds, the K_p value was set to 8, the K_i value was set to 1 and the K_d value was set to 25.

Next, for testing the effect of proportional gain, K_p , on car performance, half and ten times the "best" K_p values were tested. For half of the ideal K_p value, the system showed a slower rise time and increased steady state error. This can be seen in Fig. 7. This result indicates that the system had a reduced response to errors. For ten times the ideal K_p value, the system has a faster rise time but high overshoot. This can be seen in Fig. 8. This shows that though the system was more responsive, the results were less stable and contained excessive corrections.

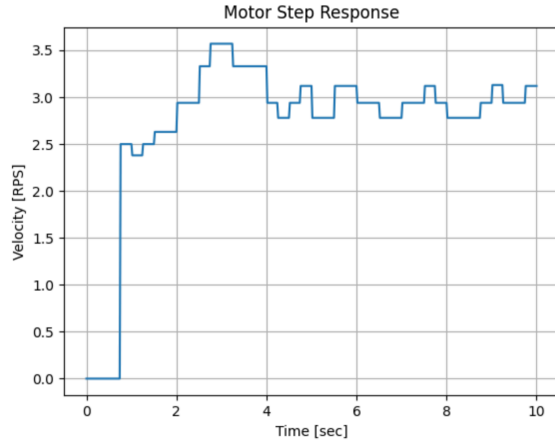


Fig. 7. Motor step response for half K_p value and no K_i value.

K_p and K_i values are important in having a stable system. The proportional gain affects the systems responsiveness to errors. By increasing the K_p value, one can decrease rise times and steady state error. However, a higher K_p value introduces a greater overshoot and less consistency in the system. Lowering K_p will result in a slower system and larger steady state error. This shows that K_p must be tuned correctly to balance the responsiveness and stability of the system.

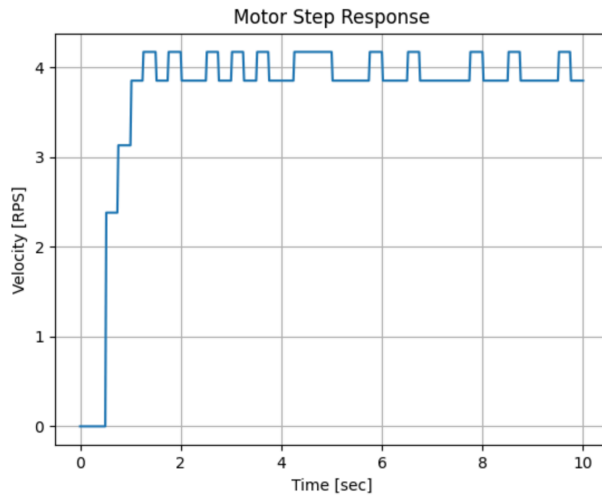


Fig. 8. Motor step response for ten times K_p value and no K_i value.

The integral gain, K_i , deals with the cumulative error over time. This improves the system's ability to deal with steady state error. Increasing K_i reduces the effects of error over the long term, however if K_i is

too high, it can lead to higher overshoot, setting time and degrade stability. K_i , therefore must be carefully implemented as not to affect the stability of the system too strongly. Together, K_p and K_i must be tuned together to create an accurate and stable control system.

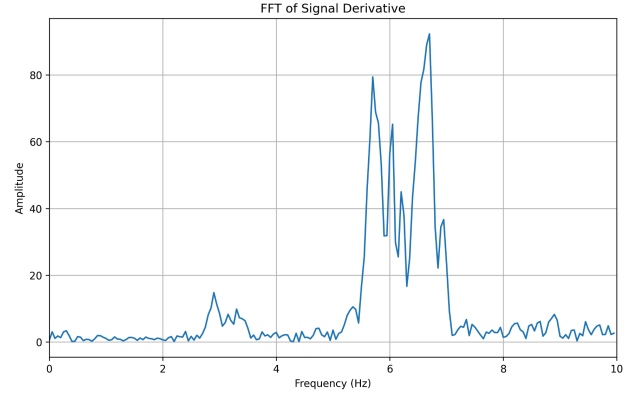


Fig. 9. FFT of dataset.

From the FFT plot (Fig. 9), we can observe a small peak at approximately 3 Hz, which corresponds to our desired speed. Moreover, at 9 Hz, there is also a small peak with a lower amplitude than that at 3 Hz. Combined, this evidence proves that the real RPS is at about 3. This indicates that our control system is accurate and robust. However, there is a significant peak at about 6 RPS. This indicates that there exists significant noise disrupting the data collection process. The error may come from the instability of the car wheels or the inconsistent room lighting. Additionally, due to faulty hardware leading to the LED by the photoresistor occasionally turning off, this error can be attributed to this issue. To improve this, a more stable wheel, reliable hardware, and room environment can be used to prevent noise.

VI. CONCLUSION

This project successfully demonstrated a Raspberry Pi-powered car capable of autonomous navigation, object tracking, and consistent speed control using PID controllers and sensor feedback. By tuning PID parameters, we achieved minimal overshoot, steady-state error, and smooth operation, as shown by the motor step response plots and the FFT analysis.

The camera-based color tracking program enabled accurate steering towards a blue or yellow

object, while the ultrasonic sensor ensured the car would slow down when it approached an object.. Despite some challenges like noise, hardware calibration and other hardware issues, these were solved with efficient sampling and consistent recalibration. Future enhancements could include more advanced computer vision algorithms and predictive control for a more efficient system.

Overall, this project demonstrated the feasibility of low cost robotic systems for real-world applications by combining PID control, camera tracking, and multi-sensor integration to achieve reliable and adaptive performance.

VII. REFERENCES

[1] McAleer, Kevin. "PID Controllers." *Kev's Robots*, www.kevsrobots.com/resources/how_it_works/pid-controllers.html. Accessed 09 Dec. 2024.